

Preprocessor



What will you learn?

- How to use `#include` for developing a large program.
- How to use `#define` to create macros and macros with arguments.
- To use conditional compilation to specify portions of a program that should not always be compiled (such as code that assists you in debugging).
- To display error messages during conditional compilation.
- To use assertions to test if the values of expressions are correct.

Introduction

- Preprocessing
 - Occurs before a program is compiled
 - Inclusion of other files
 - Definition of symbolic constants and macros
 - Conditional compilation of program code
 - Conditional execution of preprocessor directives
- Format of preprocessor directives
 - Lines begin with #
 - Only whitespace characters and comments may appear before a preprocessor directive on a line

The `#include` Preprocessor Directive

- `#include`
 - Copy of a specified file included in place of the directive
 - `#include <filename>`
 - Search standard library for file
 - Use for standard library files
 - `#include "filename"`
 - Search current directory, then standard library
 - Use for user-defined files
 - Used for:
 - Programs with multiple source files to be compiled together
 - Header file – has common declarations and definitions (classes, structures, function prototypes)

The `#define` Preprocessor Directive: Symbolic Constants

- `#define`
 - Preprocessor directive used to create symbolic constants and macros
 - Symbolic constants
 - When program compiled, all occurrences of symbolic constant replaced with replacement text
 - Format
 - `#define identifier replacement-text`
 - Example:
 - `#define PI 3.14159`
 - Everything to right of identifier replaces text
 - `#define PI = 3.14159`
 - Replaces "PI" with "`= 3.14159`"
 - **Cannot redefine** symbolic constants once they have been created

The `#define` Preprocessor Directive: Macros

- Macro
 - Operation defined in `#define`
 - A macro without arguments is treated like a symbolic constant
 - A macro with arguments has its arguments substituted for replacement text, when the macro is expanded
 - Performs a text substitution – no data type checking
 - The macro

```
#define CIRCLE_AREA( x ) ( PI * ( x ) * ( x ) )
```

would cause

```
area = CIRCLE_AREA( 4 );
```

to become

```
area = ( 3.14159 * ( 4 ) * ( 4 ) );
```

The `#define` Preprocessor Directive: Macros

- Use Parentheses

- Without them, for the macro `#define CIRCLE_AREA( x ) PI * x * x`

`area = CIRCLE_AREA(c+2)` would lead to `area = 3.14159 * c + 2 * c + 2`

- Multiple arguments

`#define RECTANGLE_AREA( x, y ) ( ( x ) * ( y ) )`

would cause

`rectArea = RECTANGLE_AREA( a + 4, b + 7 );`

to become

`rectArea = ( ( a + 4 ) * ( b + 7 ) );`

The `#define` Preprocessor Directive: Macros

- `#undef`
 - Undefine a symbolic constant or macro
 - If a symbolic constant or macro has been undefined it can later be redefined

Conditional Compilation

- Conditional compilation
 - Control the execution of preprocessor directives and the compilation of program code
 - Cast expressions, sizeof, enumeration constants cannot be evaluated in preprocessor directives

- Structure similar to if

```
#if !defined( NULL )  
    #define NULL 0  
#endif
```

- Determines if symbolic constant NULL has been defined
 - If NULL is defined, defined(NULL) evaluates to 1
 - If NULL is not defined, this function defines NULL to be 0
- Every #if must end with #endif
- #ifdef short for #if defined (name)
- #ifndef short for #if !defined (name)

Conditional Compilation

- Other statements
 - `#elif` – equivalent of `else if` in an `if` statement
 - `#else` – equivalent of `else` in an `if` statement
- "Comment out" portions of code
 - If the code contains comments, cannot use `/* ... */` **for this purpose**
 - Instead, we should use

```
#if 0
    code commented out
#endif
```
 - To enable code, change 0 to 1

Conditional Compilation

- Debugging

```
#define DEBUG 1
#ifdef DEBUG
    printf("Variable x = %d\n", x);
#endif
```

- Defining DEBUG to 1 enables code
- After code corrected, remove #define statement
- Debugging statements are now ignored

Conditional Compilation

```
#include <stdio.h>

// Define platform macros based on the target platform
#define WINDOWS

int main() {
    #ifdef WINDOWS
        printf("Running on Windows.\n");
    #elif defined(LINUX)
        printf("Running on Linux.\n");
    #else
        printf("Unknown platform.\n");
    #endif

    return 0;
}
```

#error Preprocessor Directives

- Format: `#error tokens` `#error <error_message>`
 - Tokens are sequences of characters separated by spaces
 - "I like C++" has 3 tokens
- When a `#error` directive is processed on some systems, it
 - displays a message including the specified tokens as an error message, e.g.
 `#error 1 – Out of range error`
 - stops preprocessing and prevents program compilation

```
#include <stdio.h>

// Define platform macros
#define WINDOWS // Uncomment to simulate Windows
platform

int main() {
    #if !defined(WINDOWS)
        #error "This code must be compiled on a Windows
platform."
    #endif

    printf("Running on Windows platform.\n");
    return 0;
}
```

The # and ## Operators

- #
 - Causes a replacement text token to be converted to a string surrounded by quotes
 - The statement

```
#define HELLO( x ) printf( "Hello, " #x "\n" );
```

would cause

```
HELLO( John )
```

to become

```
printf( "Hello, " "John" "\n" );
```
 - Strings separated by whitespace are concatenated when using printf

The # and ## Operators

- ##

- Concatenates two tokens

- The statement

```
#define TOKENCONCAT( x, y )  x ## y
```

would cause

```
TOKENCONCAT( O, K )
```

to become

```
OK
```


Line Numbers

- `#line`
 - Renumbers subsequent code lines, starting with the specified constant integer value
 - File name can be included
 - `#line 100 "myFile.c"`
 - Indicates that lines are numbered from 100 beginning with next source code line and that the name of the file for the purpose of any compiler messages is "file1.c"
 - Makes errors more meaningful
 - Line numbers do not appear in source file

Predefined Symbolic Constants

- Four predefined symbolic constants
 - Cannot be used in `#define` or `#undef`

Some predefined symbolic constants

Symbolic constant	Explanation
__LINE__	The line number of the current source code line (an integer constant).
__FILE__	The presumed name of the source file (a string).
__DATE__	The date the source file was compiled (a string of the form "Mmm dd yyyy" such as "Jan 19 2002").
__TIME__	The time the source file was compiled (a string literal of the form "hh:mm:ss").
__STDC__	The value 1 if the compiler supports Standard C.

Note: These identifiers and the defined identifier cannot be used in **#define** or **#undef** directives.

Assertions

- `assert` macro
 - Defined in the header `<assert.h>`
 - Tests value of an expression
 - If 0 (`false`) prints error message and calls `abort`
 - Example:

```
assert( x <= 10 );
```

If `x` is greater than 10 when the preceding statement is encountered in a program, an error containing the line number and file name is printed and the program terminates.
- If `NDEBUG` is defined
 - All subsequent `assert` statements ignored
 - **Hence, when assertions are no longer needed, the line**

```
#define NDEBUG
```

is inserted in the program file rather than deleting each assertion manually.